HPC-CCP Deliverable 02

Group: Akif Jawad (**23i-0583**) | Saim Ahmad (**23i-0781**) | Faran Azam (**23i-0075**)

Implementaion

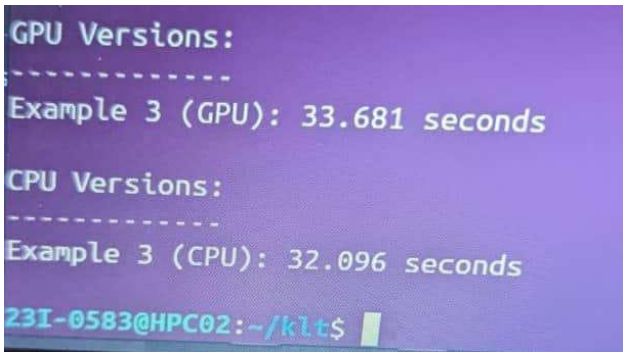
Our analysis from Deliverable 1 helped deduced that, the _convolveImageVert and _convolveImageHoriz had the largest overhead and we tried to minimize it by sending them to the GPU. Aside from these we also found that the _interpolate and the _compute2x2gradient functions were also causing a significant overhead which we tried to minimize as well. This was implemented by using different threading routes that would provide the best possible outcomes.

Data Overview

The sample data provided to us was used for testing. A larger sample data was used later on; 100 frames of HD (3840x2160) images was used for better results.

Result

Initial results were disappointing where there was no speedup (negative at times).



GPU Functions

1. **Interpolate**
2. **2by2GradientMatrix**
3. **Convolve_vert**
4. **Convolve_horiz**

Reason/Interpretation

It was found that the _interpolate function was causing large overheads due to excessive memory copy overshadowing the actual overhead.

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
72.9	1,999,755,005,375	1,239,514	1,613,338.0	169,305.0	93,996	20,915,450	1,682,118.5	cudaMemcpy2DToArray
13.2	363,420,973,187	1,239,514	293,196.3	117,312.0	30,605	20,442,114	307,671.6	cudaFreeArray
7.3	199,728,278,964	1,239,514	161,134.3	104,830.0	46,200	23,311,225	204,183.4	cudaMallocArray
4.5	123,123,654,992	1,571,750	78,335.4	2,565.0	1,307	93,656,023	184,735.4	cudaMalloc
1.0	26,583,385,544	1,571,750	16,913.2	3,415.0	1,334	21,593,666	39,073.5	cudaFree
0.7	18,304,306,809	1,571,750	11,645.8	9,077.0	2,268	11,387,156	78,601.2	cudaMemcpy
0.2	4,921,562,841	620,123	7,936.4	7,968.0	5,578	714,229	5,676.7	cudaLaunchKernel
0.2	4,724,312,714	1,239,514	3,811.4	3,612.0	2,545	887,565	7,429.8	cudaCreateTextureObject

[5/7] CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
99.8	2,036,887,530,649	1,239,514	1,643,295.3	185,505.0	168,161	20,063,841	1,670,420.7	[CUDA memcpy Host-to-Array]
0.2	3,100,410,489	1,190,660	2,603.9	1,664.0	480	10,881,391	71,104.0	[CUDA memcpy Device-to-Host]
0.1	1,133,689,781	381,090	2,974.9	640.0	352	3,952,499	90,107.6	[CUDA memcpy Host-to-Device]
0.0	412,037,566	570,537	722.2	384.0	320	32,576	545.1	[CUDA memset]

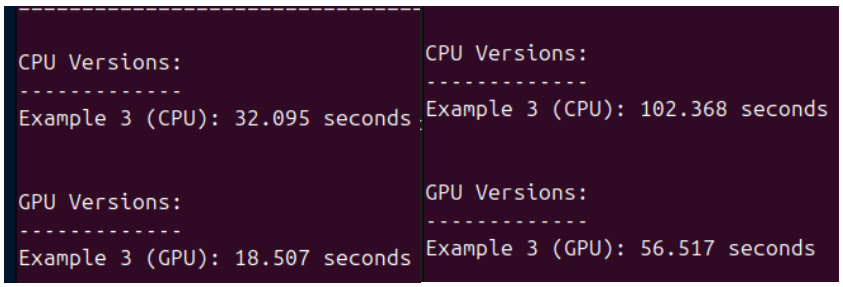
Processing [nsys_report.sqlite] with [/opt/nvidia/hpc_sdk/Linux_x86_64/25.7/profilers/Nsight_Systems/host-linux-x64/reports/cuda_gp

Readjustment

We proceeded to remove the functions with this excessive memory copying which were identified to be the _interpolate and the _compute2x2gradient functions. This adjustment proved quite beneficial with a significant difference in the before and after of the GPU time.

Result

The following results were based on the different numbers of features and image pixels of the sample



Overall time taken for execution of example 3.

```
Total memory time: 3.721 ms
Total GPU time: 0.297 ms
Total memory time: 3.701 ms
Total GPU time: 0.356 ms
Total memory time: 3.699 ms
Total GPU time: 0.354 ms
Total memory time: 3.975 ms
Total GPU time: 0.287 ms
Total memory time: 3.724 ms
Total GPU time: 0.853 ms
```

Time taken by the convolve function during each call most of the time is spent on copying data from CPU memory to GPU memory. (This will be optimized in the next versions)

Speed up

Expected Speed up (from deliverable 1):

Function	Parallel Portion (f)	Speedup (N=68)	Speedup (N=8704)	Theoretical Max
Convolution	46.15%	1.83	1.856	1.857

Computed Speed up:

Frames (3840x2160)	**CPU Time (s)**	**GPU Time**	**Speed Up**
30	32.095	18.507	1.73x
100	102.368	56.517	1.811x

To do next

- Optimising the GPU implementation of relevant functions to increase the speedup even further
- Use of shared memory to bypass the memory copying overhead of the previously identified functions.